# Úkol: RPG Souboj - Systém postav a příšer

David Marek

## Zadání

Píšete textovou RPG hru. Hráč si vybere postavu - Válečníka nebo Mága — a postupně bojuje s příšerami. Každá postava útočí jinak. Mág navíc umí léčit, Válečník se umí bránit. Hra skončí, jakmile jsou poraženy všechny příšery nebo hráč padne.

## Přehled tříd a rozhraní

- `Entity` - abstraktní rodič všeho živého
  - `Character` - abstraktní postava hráče
    - `Warrior` - implementuje `IBlockable`
    - `Mage` - implementuje `IHealable`
  - `Enemy` - abstraktní nepřítel
    - `Goblin` - slabý nepřítel
    - `Dragon` - silný nepřítel

Rozhraní:

- `IHealable` - schopnost léčení
- `IBlockable` - schopnost blokování útoku

## Jak fungují útoky

Každý útok (metoda `Attack`) zavolá `target.TakeDamage(damage)` na cíli. Metoda `TakeDamage` je definovaná v `Entity` - útočník tedy nepotřebuje vědět, zda útočí na postavu nebo příšeru, stačí mu vědět, že to je `Entity`. To, jak velká bude damage je vypočítáno v metodě `Attack` při volání `TakeDamage` podle následující tabulky.

| Kdo útočí | Poškození | Příklad výpisu |
|---|---|---|
| Warrior | `Level × 8` | `Thorin útočí mečem Ohnivý meč a způsobuje 16 poškození!` |
| Mage | `ManaPoints / 2` | `Gandalf sesílá ohnivou kouli a způsobuje 25 poškození! Zbývá 40 many.` |
| Goblin | pevně 8 | `Goblin škrábe a způsobuje 8 poškození!` |
| Dragon | pevně 25 | `Drak chrlí oheň a způsobuje 25 poškození!` |

## Krok za krokem

### Krok 1 - Abstraktní třída `Entity`

- Vlastnosti: `Name` (string), `Health` (int).
- Konstruktor nastaví jméno a životy.
- Property `IsAlive`: vrátí `true`, pokud jsou životy větší než 0.
- **Abstraktní metoda** `Attack(Entity target)` - potomci ji musí povinně implementovat.
- Metoda `TakeDamage(int damage)`: odečte `damage` od `Health` (Health nesmí klesnout pod 0) a vypíše: `[Jméno] obdrželo [damage] poškození. Zbývá [Health] HP.`

### Krok 2 - Rozhraní `IHealable` a `IBlockable`

`IHealable` - metoda `Heal(int amount)`:

- Přidá životy (nesmí přesáhnout maximum).
- Vypíše: `[Jméno] se vyléčilo o [amount] HP. Nyní má [Health] HP.`

`IBlockable` - metoda `Block()`:

- Vrátí `bool`, zda se postava úspěšně zablokovala.
- Použijte `Random.Shared.Next(maxNumber)` pro vygenerování náhodného čísla - šance na úspěšný blok je 40 %.
- Při úspěšném bloku vypíše: `[Jméno] zablokoval útok!`

  **Poznámka:** Rozhraní sama o sobě neznají `Name` ani `Health` - ty jsou v `Entity`. Třídy `Warrior` a `Mage` dědí z `Character` (a tedy i z `Entity`), takže ve svých implementacích rozhraní k těmto vlastnostem přístup mají.

### Krok 3 - Abstraktní třída `Character` dědí z `Entity`

- Přidá vlastnosti `Level` (int) a `MaxHealth` (int - potřebné pro léčení).
- Metoda `virtual void LevelUp()`: zvýší level o 1 a vypíše: `[Jméno] postoupilo na level [Level]!`

### Krok 4 - Třída `Warrior` dědí z `Character`, implementuje `IBlockable`

- Vlastnost `WeaponName` (string).
- Útok dle tabulky výše, volá `target.TakeDamage(...)`.
- Implementace `Block()` dle rozhraní.
- Override `LevelUp()`: zavolá `base.LevelUp()` a navíc vypíše: `Válečníkova síla roste!`

### Krok 5 - Třída `Mage` dědí z `Character`, implementuje `IHealable`

- Vlastnost `ManaPoints` (int). Každý útok stojí 10 many.
- Útok dle tabulky výše. Pokud mana $< 10$, útok neprovede a vypíše: `[Jméno] nemá dostatek many!`
- Implementace `Heal(int amount)`: léčení stojí 20 many. Pokud mana $< 20$, vypíše: `[Jméno] nemá dost many na léčení!`

### Krok 6 - Třídy `Enemy`, `Goblin`, `Dragon`

- `Enemy` dědí z `Entity` - žádné extra vlastnosti nejsou potřeba.
- `Goblin` a `Dragon` implementují `Attack(Entity target)` dle tabulky výše.

### Krok 7 - Simulace v `Main`

1. Vytvořte `Warrior` (jméno "Thorin", level 2, zbraň "Ohnivý meč", 100 HP).
2. Vytvořte `List<Enemy>` s Goblinem (50 HP) a Drakem (120 HP).
3. Pro každého nepřítele simulujte souboj: hráč a nepřítel se střídají v útocích, dokud jeden z nich nemá 0 HP.
4. Před každým útokem nepřítele zkontrolujte pomocí `is`, zda hráč implementuje `IBlockable` - pokud ano, zavolejte `Block()`. Vrátí-li `true`, poškození se neprovede.
5. Po každém vyhraném souboji zavolejte `LevelUp()`. Pokud je hráč `IHealable`, zavolejte také `Heal(30)`.

Kontrola rozhraní v Main:

```
bool blocked = false;
if (player is IBlockable blocker)
{
    bool blocked = blocker.Block();
}
// V tomto bodě máme v proměnné blocked true/false jestli bude útok blokován
```

### Ukázkový výstup jednoho kola souboje

```
--- Souboj s Goblin ---
Thorin útočí mečem Ohnivý meč a způsobuje 16 poškození!
Goblin obdrželo 16 poškození. Zbývá 34 HP.
Goblin škrábe a způsobuje 8 poškození!
Thorin obdrželo 8 poškození. Zbývá 92 HP.
Thorin útočí mečem Ohnivý meč a způsobuje 16 poškození!
Goblin obdrželo 16 poškození. Zbývá 18 HP.
Thorin zablokoval útok!
...
Goblin byl poražen!
Thorin postoupilo na level 3!
Válečníkova síla roste!
```

## Rozšíření pro rychlé

- Přidejte třídu `Paladin`, která dědí z `Character` a implementuje obě rozhraní - `IHealable` i `IBlockable`. Paladin útočí za `Level × 6` a léčí se zdarma (bez many).
- Přidejte rozhraní `IDescribable` s metodou `Describe()`, která vypíše podrobný popis postavy. Implementujte ho ve všech třídách hráče a na začátku hry popište vybranou postavu.
- Uložte průběh celé hry do souboru `battle_log.txt` pomocí `StreamWriter`.